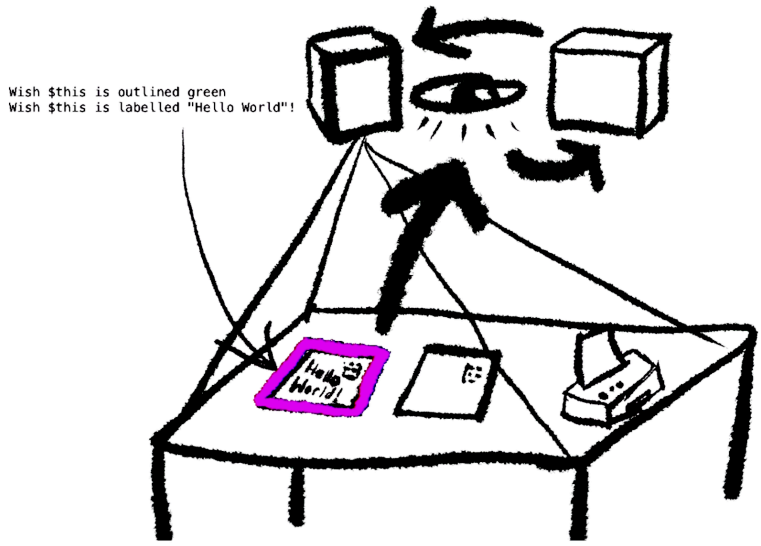
Folk Booklet

Welcome to Folk! Folk is a physical computing system: reactive database, programming environment, projection mapping. Instead of a phone/laptop/touchscreen/mouse/keyboard, your computational objects are physical objects in the real world, and you can program them inside the system itself.



This page will show you how to assemble the following pages to to create your own Folk Booklet! Through the booklet, we will walk you through examples of what Folk can do and instruct how to create your first programs. These pages will already have some pre-loaded programs to demonstrated different capabilities.

Ideally, at this point you will have -

1. Set up Folk and finished the calibration
   - [folk-WHATEVER.local:4273/calibrate]
1. Set up Printer
   - [folk.computer/guides/printer]
1. Set up Keyboard
   - [folk.computer/guides/keyboard]

Next, you will need the following -

1. This page and the following pages printed
2. Staples or other forms of binding the pages
3. Simple folding skills
4. A pair of scissors
5. Computer SSH’ed to Folk

INSTRUCTIONS on how to assemble your Folk Booklet

The following pages are to be assembled into a booklet. There are page numbers for you to follow in assembling.

Make sure to cut and fold across the appropriate lines. For pages that are folded in half, only staple the numbered page side.

Lines

Cut

Fold

FOLK BOOKLET

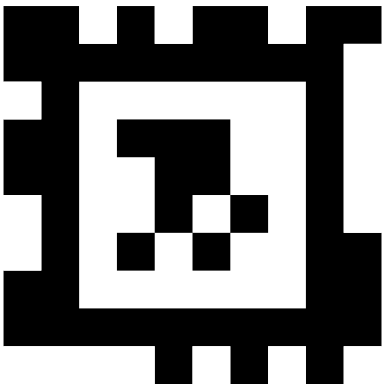
Folk Language

Folk’s language is built on TCL (pronounced ‘tickle’)

Folk is built around Tcl. We don't add any additional syntax or preprocessing to the basic Tcl language; all our 'language constructs' like When and Wish are really just plain Tcl functions that we've created. Therefore, it will eventually be useful for you to know basic TCL syntax [Include link here]

Programs

Folk sees programs through its camera and can track it in real time. Currently, Folk uses AprilTags (the QR code-like pattern), each of which are associated with a specific program stored on the computer.



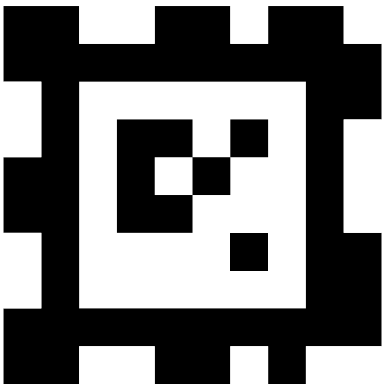
Code (45000.folk)

```
Wish $this is labelled “Welcome to Folk! I am a program”
Wish $this is outlined green
```

Try moving around this program and watch how Folk tracks your program!

Wish

Wish tells Folk what you want to see or happen - think of it like you are making a request to Folk’s projection system. Folk listens to all the wishes and makes them appear.

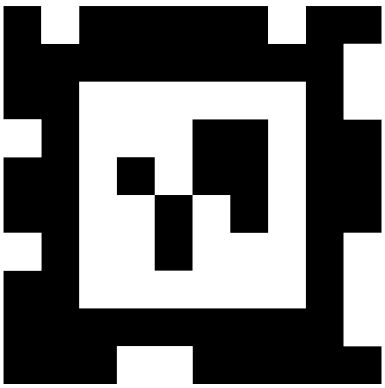


Code (45001.folk)

```
Wish $this is labelled “My wishes came true!”
Wish $this is outlined blue
```

When

When is like a rule that is always watching - it notices when things change in the physical world and reacts immediately



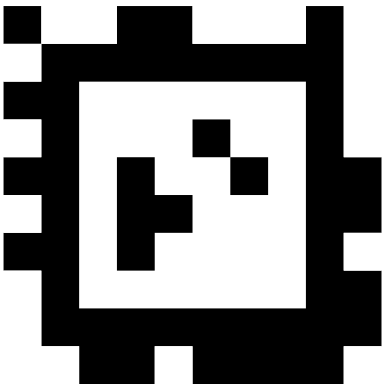
Code (45002.folk)

```
When /actor/ is cool {
  Wish $this is labelled “$actor is pretty cool”
  Wish $actor is outlined red
}

```

The inside block of then When gets executed for each claim

Claim

Claim statements assert facts into Folk’s database and lets you share information with other programs.



Code (45003.folk)

```
Claim $this is actor
Claim $this is cool


```

& (Join)

& lets you match multiple patterns together

Code

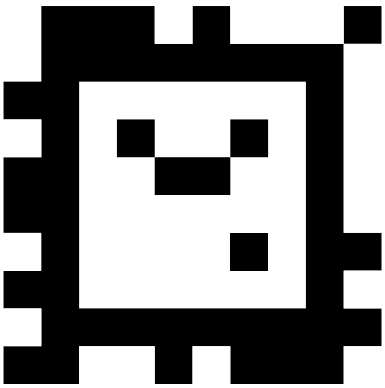
```
When /x/ is cool & \
   /x/ is a person with /n/ legs {
  Wish $this is labelled “$x is a cool person with $n legs”
}

```

You can join as many patterns as you want, separated by &

If you want to break your When onto multiple lines, remember to terminate each line with a \ so you can continue onto the next line

Animation

Using global time and different sprites, you can also create animations in Folk



Get the global clock time:

Code

```
When the clock time is /t/ {
  Wish $this is labelled $t
}
```

Use it in an animation:

Code Example

```
When the clock time is /t/ {
  Wish $this draws a circle offset [list expr {sin($t) * 50}a 0]
}
```

Now, let’s see some examples of what Folk can do!

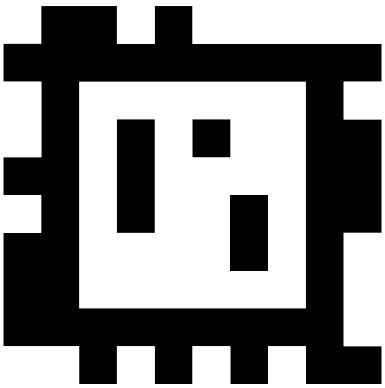
Pointers

Within Folk, programs know where they are - and what other programs are nearby.

Programs in Folk exist within a physical space. You can point one program at another, and Folk understands this spatial relationship.

With pointer programs, let’s see how two programs can interact!

Pointed Program

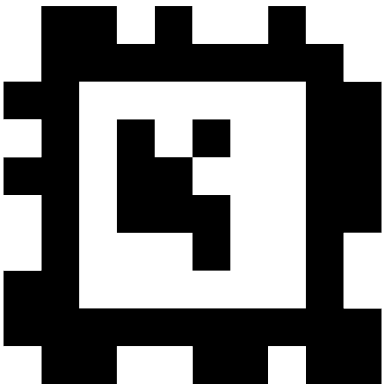


Code

```
1 Wish $this is outlined red
2 Wish $this is labelled “I am a program”
3
4
5
```

When this program is being pointed at, the pointer will display “Pointing at [program]”.

Pointing Program



Code

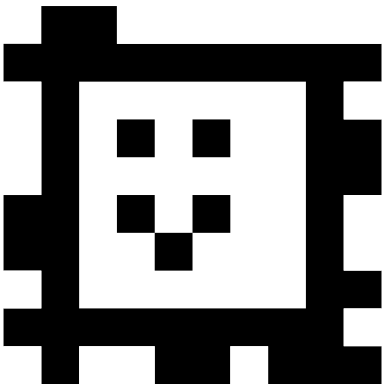
```
1 Wish $this is outlined blue
2 When $this points up at /p/ {
3   Wish $this is labelled \
4     “(Pointing at $p)”
4   Wish $p is labelled “and I am being pointed at”
5 }
```

Each program will have an associated AprilTag and program number. Other programs can locate which stored program it is pointing to on Folk.

[Fold this page out to reveal]

Camera Slice

Folk can capture and display what its camera sees



With Folk, you can create a "slice" - a live snapshot of what the camera is looking at. Point a camera slice program at any area of your table, and Folk will project that view back onto the table. It's like having a mirror that shows you what the system sees.

Code

```
When this has camera slice /slice/ {
  Wish $this displays camera slice $slice
}


```

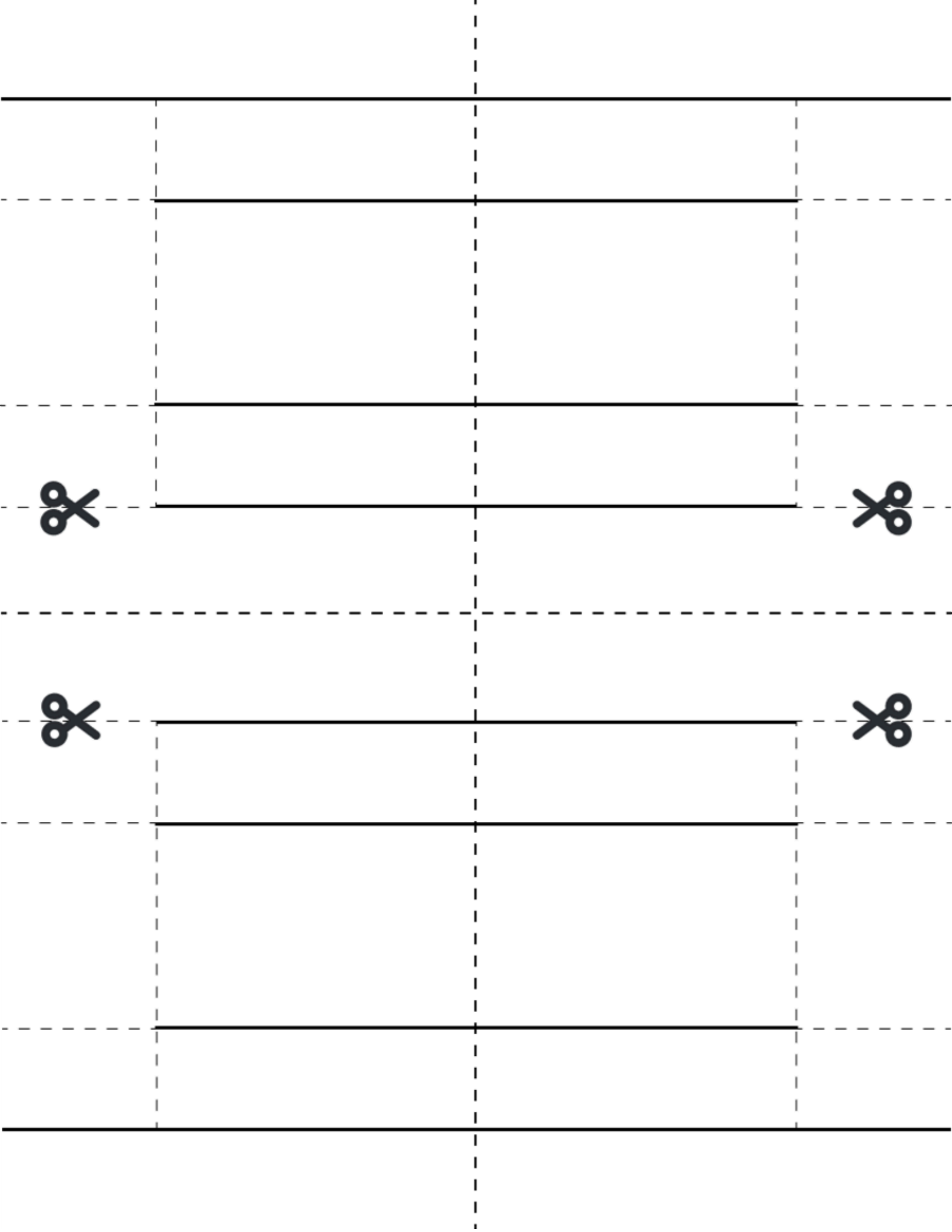

Button

Let’s make a button!

In Folk, you can create physical buttons by folding paper in a special way. When you press the button, it brings the two halves of an AprilTag together - Folk sees this and triggers your program. It's like having a real button, but made entirely of paper!

See the page for the button template. Make sure to follow along on the link below on how to create a button!

[https://folk.computer/guides/button]



Resources

Folk Computer is an open source project led by Omar Rizwan and Andrés Cuervo, and is based in Brooklyn, New York.

You can find us here!

- Folk Wiki [folk.computer]
- GitHub [github.com/folkcomputer]
- Substack / Newsletter [folkcomputer.substack.com]
- Discord [request discord access at cwervo@folk.computer]

Folk also hosts monthly open houses and welcomes visitors. Make sure to reach out to us if you're interested in visiting us at our studio.